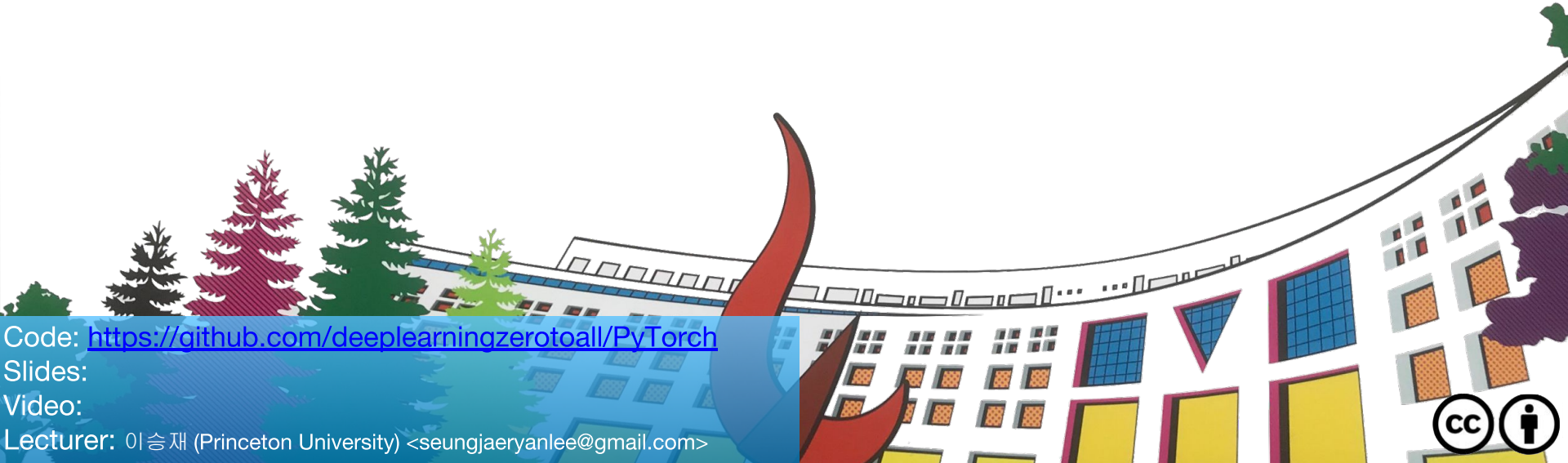
ML/DL for Everyone Season2

with PYTORCH

Linear Regression



Code: https://github.com/deeplearningzerotoall/PyTorch
Slides:
Video:
Lecturer: 이승재 (Princeton University) <seungjaeryanlee@gmail.com>

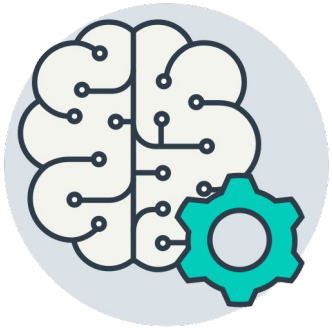
다루고자 하는 주제

- Data definition
- Hypothesis
- Compute loss
- Gradient descent

Data definition

What would be the grade if I study 4 hours?

*잘 예측하는 하나의 직선을 찾는 것 (오차 허용 가능성)

4 hours →  → ? points

Prediction

Hours (x)	Points (y)
1	2
2	4
3	6
4	?

Training dataset

Test dataset

Data definition

```
x_train = torch.FloatTensor([[1], [2], [3]])
y_train = torch.FloatTensor([[2], [4], [6]])
```

$$X_{\text{train}} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \qquad Y_{\text{train}} = \begin{pmatrix} 2 \\ 4 \\ 6 \end{pmatrix}$$

Hours (x)	Points (y)
1	2
2	4
3	6

- 데이터는 `torch.tensor`!
- 입력 따로, 출력 따로!
  - 입력: `x_train`
  - 출력: `y_train`
  - 입출력은 x, y 로 구분

Hypothesis

$$y = Wx + b$$

Weight

Bias

Hypothesis

```
x_train = torch.FloatTensor([[1], [2], [3]])
y_train = torch.FloatTensor([[2], [4], [6]])
```

```
W = torch.zeros(1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)
hypothesis = x_train * W + b
```

- Weight 와 Bias 0으로 초기화
  - 항상 출력 0을 예측
- `requires_grad=True`
  - 학습할 것이라고 명시

$$y = Wx + b$$

Compute loss

Mean Squared Error (MSE)

$$cost(W, b) = \frac{1}{m} \sum_{i=1}^{m} \left(H(x^{(i)}) - y^{(i)} \right)^2$$

*비용 함수

Mean

Prediction

Target

Compute loss

```
x_train = torch.FloatTensor([[1], [2], [3]])
y_train = torch.FloatTensor([[2], [4], [6]])
```

```
W = torch.zeros(1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)
hypothesis = x_train * W + b
```

```
cost = torch.mean((hypothesis - y_train) ** 2)
```

- `torch.mean` 으로 평균 계산!
- 한 줄인데 읽기 편한 코드 :)

$$\frac{1}{m}\sum_{i=1}^{m}\left(H(x^{(i)})-y^{(i)}\right)^2$$

Gradient descent

```
x_train = torch.FloatTensor([[1], [2], [3]])
y_train = torch.FloatTensor([[2], [4], [6]])
```

```
W = torch.zeros(1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)
hypothesis = x_train * W + b
```

```
cost = torch.mean((hypothesis - y_train) ** 2)
```

```
optimizer = optim.SGD([W, b], lr=0.01)


optimizer.zero_grad()
cost.backward()
optimizer.step()
```

- `torch.optim` 라이브러리 사용
  - `[W, b]` 는 학습할 tensor들
  - `lr=0.01` 은 learning rate

- 항상 붙어다니는 3줄
  - `zero_grad()` 로 gradient 초기화
  - `backward()` 로 gradient 계산
  - `step()` 으로 개선

Full training code

```
x_train = torch.FloatTensor([[1], [2], [3]])
y_train = torch.FloatTensor([[2], [4], [6]])

W = torch.zeros(1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)

optimizer = optim.SGD([W, b], lr=0.01)

nb_epochs = 1000
for epoch in range(1, nb_epochs + 1):
    hypothesis = x_train * W + b
    cost = torch.mean((hypothesis - y_train) ** 2)

    optimizer.zero_grad()
    cost.backward()
    optimizer.step()
```

한번만

1. 데이터 정의
2. Hypothesis 초기화
3. Optimizer 정의

반복!

1. Hypothesis 예측
2. Cost 계산
3. Optimizer 로 학습

What's Next?

- How does gradient descent minimize cost?

```
import torch

x_train = torch.FloatTensor([1],[2],[3])
y_train = torch.FloatTensor([2],[4],[6])


# torch.nn.Linear(3, 1) : 랜덤으로 W와 b를 생성
W = torch.zeros(1,requires_grad=True)   # tensor([0.]) / 특징에 따라 zeros tensor의 갯수가 늘어남
b = torch.zeros(1,requires_grad=True) # 기울기를 추적할 수 있도록 설정 / 편향은 언제나 1개이면 됨

optimizer = optim.SGD([W,b],lr=0.01)

np_epochs = 1000
for epoch in range(1,nb_epochs+1):
  hypothesis = x_train * W + b  # @로 행렬 곱을 표현해줘야함 (차원이 높아질 수록)
  cost = torch.mean((hypothesis-y)**2)

  optimizer.zero_grad() # 이전 epoch의 기울기 초기화
  cost.backward() # 손실을 미분해서 기울기 구하기
  optimizer.step() # W 가중치 업데이트
```